

DOCTOR STRANGE

Documentación de la Lógica Matemática

Álgebra Lineal — Espacios Vectoriales

Este documento describe el funcionamiento interno del programa DoctorStrange, una herramienta para analizar conjuntos de vectores, polinomios y matrices en distintos espacios vectoriales. Se explica la lógica matemática implementada, los métodos utilizados y la evolución del código desde su versión inicial hasta la versión actual corregida y mejorada.

1. Arquitectura del Proyecto

Módulo	Archivo	Responsabilidad
Vectores R■	logica/vectores.py	Producto punto, GS, diagnóstico para R■
Polinomios	logica/polinomios.py	Representación y diagnóstico en P■
Matrices	logica/matrices.py	Producto de Frobenius, GS para M■■■
Interfaz	interfaz/	Pestañas CustomTkinter + componentes visuales

2. Módulo de Vectores — logica/vectores.py

2.1 Diagnóstico del conjunto

Dado un conjunto de k vectores en ■■ representado como una matriz de $k \times n$, se calculan las siguientes propiedades:

Propiedad	Criterio matemático	Implementación NumPy
Rango	$\text{rank}(A)$	<code>np.linalg.matrix_rank(A)</code>
Lin. Independiente	$\text{rank} = k$ (núm. vectores)	<code>rango == num</code>
Generador	$\text{rank} = n$ (dim. espacio)	<code>rango == dim</code>
Base	LI AND Generador	<code>li and gen</code>
Ortogonal	$\mathbf{v}_i \cdot \mathbf{v}_j = 0 \quad \forall i \neq j$	<code>proseso_ortogonal(...)</code>
Ortonormal	Ortogonal AND $\ \mathbf{v}_i\ = 1 \quad \forall i$	<code>proseso_ortonormal(...)</code>

2.2 Verificación de ortogonalidad

Se verifica que el producto punto entre todo par de vectores distintos sea cero. Se usa `np.isclose` en lugar de comparación exacta para tolerar errores de punto flotante ($\epsilon \approx 1 \times 10^{-10}$):

```
def proseso_ortogonal(conjunto_vectores):
    n = len(conjunto_vectores)
    for i in range(n):
        for j in range(i + 1, n):
```

```

        producto = np.dot(conjunto_vectores[i], conjunto_vectores[j])
        if not np.isclose(producto, 0):
            return False
    return True

```

2.3 Gram-Schmidt — Ortogonalización

El proceso de Gram-Schmidt toma un conjunto LI y produce un conjunto ortogonal equivalente. Para cada vector v_i se proyecta sobre todos los vectores ya orthogonalizados y se resta la componente paralela:

$$w_i = v_i - \sum_{j=1}^i (v_i \cdot u_j / \|u_j\|^2) \cdot u_j$$

Corrección clave aplicada: la versión original no verificaba si el denominador $\|u_j\|^2$ era cero (división por cero cuando el conjunto era linealmente dependiente). La versión actual incluye la guarda:

```

def orthogonalizar(conjunto_vectores):
    dim = np.array(conjunto_vectores[0]).shape[0]
    ortogonales = []
    for v in conjunto_vectores:
        w = np.array(v, dtype=float)
        for u in ortogonales:
            norm_sq = np.dot(u, u)
            if not np.isclose(norm_sq, 0): # guarda vs div/0
                w = w - (np.dot(w, u) / norm_sq) * u
        if not np.allclose(w, 0): # descarta vec. cero
            ortogonales.append(w)
    return np.round(np.array(ortogonales), decimals=10)

```

2.4 Gram-Schmidt — Ortonormalización

Tras orthogonalizar, cada vector se divide entre su norma para obtener norma = 1. También se añadió guarda contra norma ≈ 0 :

```

def ortonormalizar(conjunto_vectores):
    ortogonales = orthogonalizar(conjunto_vectores)
    ortonormales = []
    for v in ortogonales:
        norma = np.linalg.norm(v)
        if not np.isclose(norma, 0): # guarda vs div/0
            ortonormales.append(v / norma)
    return np.round(np.array(ortonormales), decimals=10)

```

Corrección en ortonormal original: había un bug de indentación — el `return True` estaba dentro del `for`, haciendo que la función retornara tras verificar solo el primer vector.

3. Módulo de Polinomios — logica/polinomios.py

Los polinomios de grado $\leq n$ se representan como vectores de coeficientes en $\mathbb{R}^{n+1}$ en orden **ascendente** internamente ($c_0, c_1, \dots, c_n$). El usuario los ingresa en orden descendente ($c_n, \dots, c_0$) y la interfaz los invierte automáticamente con `matriz[:, ::-1]`.

3.1 Producto interno en P_n

El producto interno entre dos polinomios p y q representados como vectores de coeficientes es el producto punto estándar de $\mathbb{R}^{n+1}$ (heredado de `vectores.py`):

$$\langle p, q \rangle = p \cdot q = \sum_{i=0}^n p_i \cdot q_i$$

Esto es equivalente a la integral $\int_{-1}^1 p(x)q(x)dx$ solo para la base monomial estándar. Para el propósito del curso se usa el producto punto de los coeficientes.

3.2 Diagnóstico

Idéntico al de vectores: se construye la matriz de coeficientes ($k \times (n+1)$) y se aplica rank para determinar LI, generador y base. La dimensión del espacio P_n es $n+1$.

3.3 Conversión a notación matemática

La función `vector_a_polinomio(coefs)` convierte el vector de coeficientes a una cadena legible, usando superíndices Unicode para las potencias. Omite términos con coeficiente ≈ 0 y simplifica ± 1 como prefijo sin el número:

```
# Ejemplo de salida:
# coefs = [1, 0, -2, 1] (ascendente: 1 + 0x - 2x^2 + x^3)
# resultado: "x^3 - 2x^2 + 1"
```

3.4 Ortogonalización y ortonormalización

Usa directamente las funciones de `logica/vectores.py` (importadas), ya que matemáticamente el proceso es idéntico al aplicar Gram-Schmidt sobre los vectores de coeficientes.

4. Módulo de Matrices — `logica/matrices.py`

El espacio $M_{n \times n}$ de matrices reales de dimensión $m \times n$ es isomorfo a $\mathbb{R}^{m \times n}$. Sin embargo, las matrices se tratan como objetos 2D para usar la norma y el producto interno correctos.

4.1 Producto interno de Frobenius

El producto interno natural en $M_{n \times n}$ es el producto de Frobenius:

$$\langle A, B \rangle_F = \text{tr}(A^T B) = \sum_i \sum_j A_{ji} \cdot B_{ji}$$

Implementado como `np.sum(A * B)` (multiplicación elemento a elemento y suma). La versión original usaba `np.vdot(A, B)` que aplana los arrays internamente — aunque da el mismo resultado numérico, semánticamente es incorrecto para matrices.

```
def producto_interno(A, B):
    return np.sum(A * B) # Frobenius explícito
```

4.2 Norma de Frobenius

La norma asociada al producto de Frobenius es:

$$\|A\|_F = \sqrt{\langle A, A \rangle_F} = \sqrt{\sum_i \sum_j A_{ji}^2}$$

Calculada con `np.linalg.norm(M, 'fro')`. Esta es la norma que se muestra en la interfaz para cada matriz del proceso GS.

4.3 Gram-Schmidt en $M_{n \times n}$

El proceso es análogo al de vectores, usando el producto de Frobenius en lugar del producto punto:

$$W_i = M_i - \sum_j (\langle M_i, U_j \rangle_F / \langle U_j, U_j \rangle_F) \cdot U_j$$

Las mismas guardas contra norma cero se aplican. Cada matriz se descarta si el resultado es la matriz nula (indicando dependencia lineal).

4.4 Diagnóstico de matrices

Para calcular rango e independencia lineal, cada matriz $m \times n$ se aplanar a un vector de $m \cdot n$ componentes y se forma la matriz de diagnóstico de $k \times (m \cdot n)$:

```
def diagnosticar_matrices(arr, m, n_val):
    dim_espacio = m * n_val          # dim(Mmxn) = m·n
    num = arr.shape[0]               # número de matrices
    rango = np.linalg.matrix_rank(
        arr.reshape(num, dim_espacio) # aplanar cada M a [ ]
    )
    li = rango == num
    gen = rango == dim_espacio
    base = li and gen
```

5. Resumen de Correcciones vs. Versión Original

Bug	Versión original	Versión corregida
División por cero en GS	Sin guarda → crash cuando el conjunto es LI	Verifica $norm_sq \approx 0$ antes de dividir
Vector cero en GS	Agrega el vector cero → GS subsiguiente se desbarra	Descarta con <code>np.allclose(w, 0)</code>
ortonormal retornaba pronto	return True dentro del for → chequeaba solo el True fuera del for	return True fuera del for
np.vdot para matrices	np.vdot aplanar internamente, semántica incorrecta	<code>np.sum(A * B)</code> — Frobenius explícito
GS solo si es base	Se ocultaba GS cuando no era base, incluso si era LI	GS siempre muestra siempre que el conjunto sea LI
Orden de coeficientes	Polinomios en orden ascendente en el input	usuario ingresa descendente, se invierte internamente

6. Flujo completo de ejecución

El siguiente diagrama describe el flujo que sigue el programa al presionar 'Diagnosticar' en cualquiera de las tres pestañas:

Paso	Acción
1	Leer input del usuario (texto) y parsear a array NumPy
2	Validar dimensiones (num componentes == n declarado)
3	Calcular rango con <code>np.linalg.matrix_rank</code>
4	Determinar LI, Generador, Base según rango
5	Verificar ortogonalidad (producto interno par a par ≈ 0)
6	Verificar ortonormalidad (ortogonal AND normas ≈ 1)
7	Mostrar tabla de propiedades en tarjeta visual
8	Si LI: ejecutar Gram-Schmidt → mostrar q_i con normas
9	Si LI: ortonormalizar → mostrar e_i con normas (deben ser = 1)